Model Adaptation

From an Off-the-Shelf Model to *Your* Model

Matt Redmond

15 Apr 2026

Where this fits

You can already get a lot from a general-purpose model with good prompts and tools.

Today's focus

What people do when they need the model to **reliably** behave a certain way—not just occasionally—for a product, a team, or a domain.

This is famously a pretty heavy-on-the-math topic. We will try to keep the math light (where possible) and the intuition strong: **what changes**, **why it helps**, and **what can go wrong**.

Learning objectives

By the end of this session, you should be able to:

1. explain the difference between **prompting** and **training** as adaptation strategies
2. describe **supervised fine-tuning (SFT)** in plain language (examples in, behavior out)
3. explain why **LoRA** tends to be the default way teams adapt large models without retraining everything
4. recognize what **RL-style fine-tuning** is trying to optimize (preferences, rules, rewards)
5. name some common failure modes: bad data, overfitting, "reward hacking," and safety issues

How we will spend the time today

1. **Framing:** what "adaptation" means in products
2. **Supervised fine-tuning:** imitation learning from examples
3. **LoRA:** efficient adaptation without replacing the whole model
4. **RL fine-tuning:** rewards, preferences, and the token-level policy picture
5. **Putting it together:** typical pipelines and evaluation
6. **Demo:** a tiny "style" adaptation on an open model
7. **Responsible use:** what to watch for before shipping

Audience-friendly promise

You do not need to memorize equations, but... You **do** want a clear mental model for decisions and conversations with other engineers.

Outline

The core situation

- A foundation model is trained on broad internet-scale text: general knowledge, mixed styles, mixed quality.
- Your organization needs something narrower: **your tone**, **your policies**, **your domain**, **your format**.
- There is an intermediate space between general purpose datasets (broad) and RAG over your specific data (narrow).

Adaptation

Any deliberate process that moves model behavior from "general" toward "fit for our use case."

What people usually want (examples)

- **Brand voice:** marketing copy that sounds like the company, not like generic AI
- **Support style:** empathetic, policy-compliant answers with consistent structure
- **Domain language:** legal, medical, finance—with correct terminology and cautions
- **Specific safety posture:** fewer unsafe completions under adversarial prompts

Important

Adaptation is not magic. If the requirement is unclear, the data is messy, or evaluation is weak, the system will still fail in production.

Three levels of "adaptation" (mental model)

Level A: Prompting + tools + context engineering Fastest to try. Changes behavior **without** updating model weights.

Level B: Training on examples (SFT) Teach the model patterns from curated input–output pairs.

Level C: Optimization beyond imitation (RL / preferences) Tune behavior using scores, rules, or human rankings.

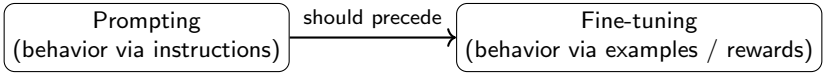
Prompting vs training: the one-slide version

Prompting

- Changes instructions at runtime
- No need to modify weights at all
- Great for exploration
- Can be brittle or hard to make targetted changes for behavior

Training (fine-tuning)

- Bakes patterns into weights (often small adapters)
- Costs data + compute + evaluation
- Can improve consistency
- Can also bake in mistakes



Outline

Supervised fine-tuning in one sentence

Show the model many examples of the behavior you want, and update it so it becomes more likely to produce similar outputs in similar situations.

Think of it like **imitation learning**: the model learns a mapping from inputs to targets you provide.

What “data” looks like (conceptually)

Most of the time, data is essentially "multi-turn chats", but serialized into strings. Common formats:

- **Instruction → response:** a user question and an ideal assistant answer
- **Multi-turn chats:** a conversation transcript with the assistant doing the right thing at each turn
- **Task-specific pairs:** e.g., messy note → clean summary in your house style

The key quality lever is usually not “more rows,” it is **better examples** and **clear evaluation**.

What improves SFT outcomes?

- **Consistency:** one style, one policy, one format—not a mixture of conflicting examples
- **Coverage:** examples that match what users will actually do (including edge cases)
- **Clean labels:** if the target answer is wrong, the model learns to be wrong confidently
- **Held-out evaluation:** questions the model has not memorized, to detect overfitting

Overfitting (intuition)

Overfitting

The model memorizes training examples but does not generalize to new, realistic inputs.

Signs include: great loss scores on the training set, but weak performance on fresh prompts, or brittle behavior when wording changes slightly.

Mitigations: more diverse data, regularization, better evaluation sets, and sometimes stopping training earlier.

SFT: what it is good / not good for

Good for:

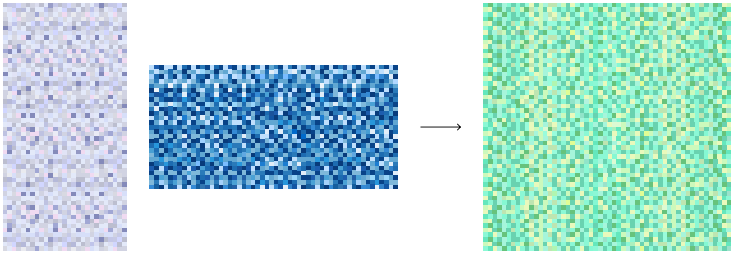
- formatting, tone, domain phrasing
- teaching a repeatable process or workflow (templates, checklists)
- reducing the need for enormous prompts

Not a substitute for:

- retrieval when facts change frequently (often combine with RAG)
- guarantees about truth (still verify externally for high-stakes domains)
- solving fundamentally underspecified questions or requirements

Outline

LoRA: cover illustration (Thinking Machines Lab)



Hero illustration from Schulman, “LoRA Without Regret” (Thinking Machines Lab). Source: `/blog/lora/svgs/lora-cover.svg`.

Why not “retrain the whole model”?

Full fine-tuning updates **all** weights. That can be:

- expensive in compute and memory
- harder to iterate quickly
- riskier if you want to keep general capabilities intact

In practice, many teams use **parameter-efficient** methods instead.

LoRA: the elevator pitch

LoRA (Low-Rank Adaptation) trains a small set of additional weights (an "adapter") instead of rewriting the entire network.

Mental picture

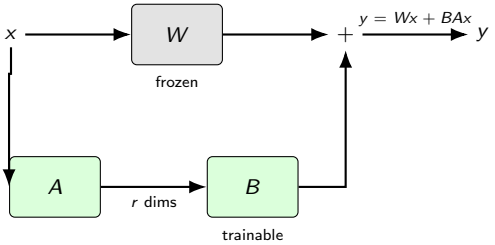
Keep the big pretrained model frozen (or mostly frozen), and learn a compact "delta" that steers outputs toward your dataset.

This is why LoRA shows up everywhere: it is the sweet spot between **cost** and **control** for many adaptation tasks.

For a practitioner-oriented take on *when* LoRA matches full fine-tuning (and why it is so common in post-training), see Schulman, "LoRA Without Regret" (Thinking Machines Lab, 2025).

LoRA: forward pass on one linear layer

Freeze the pretrained weights W. Train small matrices A and B so the effective map is $x \mapsto Wx + BAx$ (many implementations scale BA by α/r for stable step sizes).

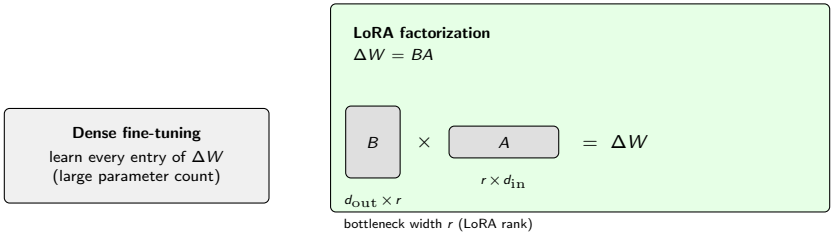


Human intuition

A sends activations through a narrow **bottleneck** of width r; B maps back to the full layer width. Only the green path is trained on your data—the frozen W keeps most pretrained behavior in place.

LoRA: why “low rank” (matrix picture)

Instead of learning a full ΔW with $d_{\text{out}} \times d_{\text{in}}$ entries, LoRA uses $\Delta W = BA$ with rank at most $r \ll \min(d_{\text{in}}, d_{\text{out}})$.



Trainable parameter count for this layer scales like $\mathcal{O}(r\,(d_{\text{in}} + d_{\text{out}}))$ instead of $\mathcal{O}(d_{\text{in}}\, d_{\text{out}})$.

Reference: Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” `arXiv:2106.09685`.

Human intuition

Instead of learning an arbitrary tweak to a gigantic matrix, you learn updates that live in a *thin* subspace—often enough to steer style or domain without touching every weight.

What you ship (conceptually)

- **Base model weights:** unchanged open-weight checkpoint (hosted by you or a vendor)
- **Adapter weights:** small files that encode your customization
- **Serving options:** load adapter at runtime, merge adapters for deployment, or route requests to different adapters

For non-technical stakeholders: think "base engine + tunable module," not "train a new engine from scratch."

Outline

When imitation is not enough

Sometimes the "right answer" is hard to write as a single gold response, but easy to **score** after the fact:

- **Preferences:** humans rank two model outputs (A better than B) for the same prompt
- **Automated checks:** unit tests pass, JSON validates, policy rules fire correctly
- **Lightweight rewards:** length within bounds, citation present, tool call succeeded

That is where **RL-style** training and **preference optimization** enter: optimize a *policy* (the model) using signals richer than "match this exact string."

What is reinforcement learning?

- An **actor** repeatedly **chooses actions** in an **environment**.
- After each action, the environment yields a new **situation** (often summarized by a **state**) and sometimes a numeric **reward**.
- The actor learns a **policy**: a rule for choosing actions to maximize **long-run return** (a weighted sum of rewards, often with **discounting** so sooner rewards count more).

Contrast with supervised learning

In supervised learning, targets are given up front. In RL, learning is driven by **interaction**: try behaviors, observe outcomes, update the policy.

Markov Decision Processes (MDPs)

A common formal starting point is an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- $\mathcal{S}$: possible **states** (what the actor is allowed to know about the situation)
- $\mathcal{A}$: **actions** (what the actor can do)
- $P(s' \mid s, a)$: how the environment evolves (**Markov** property: the next state depends on the current state and action, not the full history)
- r: **reward** when a transition happens (may be sparse—zeros most steps)
- $\gamma \in [0, 1)$: **discount** that trades off soon vs. later rewards

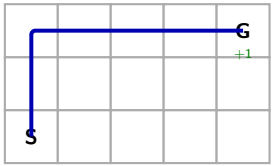
Goal: find a policy $\pi(a \mid s)$ that maximizes expected discounted return, $\mathbb{E}\left[\sum_{t \geq 0} \gamma^t r_t\right]$.

Human intuition

"Markov" means the future depends on the present state and action, not on the whole backstory—a modeling choice that keeps the math tractable. Discount γ captures "care about later outcomes, but not infinitely far ahead."

Toy problem 1: tiny grid navigation

- **States:** cells on a small grid ("where am I?")
- **Actions:** move up / down / left / right (stay inside the grid)
- **Reward:** $+1$ when you first enter the goal; elsewhere 0 unless we add hazards or bonuses (next slide)



Blue = one shortest path (six up/right moves, then +1 at G)

Why this builds intuition

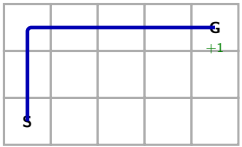
Good behavior is a **trajectory**: many dependent decisions. Credit is shared across the path—not "one label per cell," but experience of what tends to reach the goal.

Open 5×3 grid: $S = (0, 0)$, $G = (4, 2)$. Shortest routes need six steps; the blue path goes along the *top* corridor through $(2, 2)$.

Toy problem 1: hazards and bonuses reshape "best"

Keep the same grid. Add a **sinkhole** at $(2, 2)$: entering it ends the episode with reward $-\infty$ (in practice: "unacceptable—never do this").

Before: only $+1$ at G



shortest path uses (2, 2)

After: sink at $(2, 2)$, chest at $(3, 0)$

dashed = old route hits sink orange = safe detour

Treasure: put $+0.4$ the first time you enter $(3, 0)$. The orange detour already passes through $(3, 0)$, so total return becomes $+1+0.4$ (still six steps)—whereas any path that tried to stay "high" to avoid the sink would have to reroute and might *miss* the chest. With a small per-step penalty, you might prefer a shorter safe path without treasure; the point is that **return is a sum over the whole trajectory**, not just "distance to G."

Human intuition

Reward design picks which trajectories look optimal. Change the hazards or bonuses, and the best path can move—same idea when the "grid" is token choices and the rewards are rules, tests, or preference scores.

Horizons: why language generation "feels like RL"

- **Composition:** early actions change what good later actions look like (same as navigation).
- **Sparse rewards:** the meaningful signal may arrive only after a long sequence (like reaching a goal).
- **Stochastic policies:** randomness can help **exploration** (trying actions to discover what works).

Autoregressive text generation is the same broad pattern: many sequential decisions (tokens), with evaluation that may only make sense once a full candidate answer exists.

Mapping an autoregressive LLM onto the RL picture

RL idea	Typical LLM fine-tuning view
Policy π	Next-token distribution: $p_\theta(\cdot \mid \text{prompt, tokens so far})$
State	The current prefix (prompt plus generated tokens); practically what conditions the next logits
Action	Generate one token from a large but finite vocabulary
Environment	Mostly **deterministic**: append the token, advance one step; stop at EOS or a length limit
Reward	$R(x, y)$ from rules, humans, or learned models—often only after a full completion y

The next slide uses this same mapping with the vocabulary we will keep for the rest of the unit.

Human intuition

In LLM fine-tuning the "environment" is mostly bookkeeping (append a token, advance). The interesting parts are the **policy** (next-token logits) and whatever produces **reward** once a full answer exists.

RL vocabulary (first pass)

Environment Whatever produces the next situation the agent faces. For text generation, it is mostly **deterministic**: you append the chosen token to the prefix.

State A summary of "where we are." Practically: the **prompt plus tokens generated so far** (often represented by KV-cache internals, but the conceptual state is the prefix).

Action One step of what the agent chooses. For an autoregressive LM: **which token to emit next** from a finite vocabulary.

Reward A number scoring how good things turned out. May arrive only at the end of a completion, or at intermediate checks.

Policy A rule for choosing actions given states. Here: **conditional probabilities over next tokens**.

The policy is a distribution over *next tokens*

Fix model parameters θ. After prompt x, suppose the generated prefix is $y_{<t} = y_1, \ldots, y_{t-1}$.

Token-level policy

The learned policy is

$$\pi_\theta(a \mid s_t) \;=\; \Pr_\theta(\text{next token} = a \mid x,\, y_{<t}),$$

where a ranges over the model vocabulary $\mathcal{V}$ (tens of thousands of tokens).

So "policy" is not an abstract game-theory object—it is literally the model's **softmax over vocabulary** at the current position. A full answer is built by **sampling or greedily choosing** one action after another until a stop condition.

Human intuition

Nothing mystical: "policy" here is the same next-token distribution you would inspect in any notebook; RL just uses rewards to nudge those probabilities toward better full answers.

Trajectory = a sequence of token actions

A **trajectory** (one rollout) is a sequence of actions $\tau = (a_1, \ldots, a_T)$ that produces completion $y = a_1 \cdots a_T$.

Under the usual autoregressive factorization,

$$\pi_\theta(\tau \mid x) \;=\; \prod_{t=1}^{T} \pi_\theta(a_t \mid x,\, a_{<t}).$$

The probability of an entire answer is the product of the probabilities of each next-token choice along the way.
RL fine-tuning adjusts θ so that **high-reward trajectories become more probable**.

Human intuition

Multiplying step probabilities is "every token choice had better be plausible": one terrible token can tank the whole trajectory, which is why credit assignment is hard.

Example: sparse end reward (code generation)

Setup: prompt x asks for a function; the model generates code y until end-of-sequence.

- **State at step t:** $(x, a_{<t})$ — partial program so far
- **Action:** next token (identifier, bracket, newline, ...)
- **Reward:** run tests *after* generation finishes:

$$R(x, y) = \begin{cases} 1 & \text{if all unit tests pass,} \\ 0 & \text{otherwise.} \end{cases}$$

Training should increase $\pi_\theta(y \mid x)$ for completions that pass tests.
Credit assignment is nontrivial: many token choices contributed; RL methods estimate which increases in probability help most.
This is the sparsest possible learning signal - much research being done on how to do better reward shaping during the rollout.

Human intuition

Like getting a single pass/fail after a long free-response exam: a strong signal, but blunt—you need algorithms that infer which early decisions actually moved the needle.

Example: preference pair (no numeric reward yet)

For the same prompt x, sample two completions $y^{(1)}, y^{(2)}$ from $\pi_\theta(\cdot \mid x)$ (possibly at higher temperature).

A human (or stronger model) labels which is better: $y^{(1)} \succ y^{(2)}$.

What this is: a **relative** signal—"A is better than B for this x"—not a number attached to each completion in isolation. You do not yet know *how much* better, or either answer's absolute quality; you only learn a local piece of an ordering.

Many such pairs (across prompts and pairs) still **constrain** what "good" means and can be aggregated into a model of preferences. Classic pipelines fit a **reward model** $\hat{R}_\phi(x, y)$ from pairs, then run RL on $\hat{R}$; **DPO** updates θ directly from chosen/rejected pairs without fitting $\hat{R}$ or on-policy rollouts.

Human intuition

People are often better at saying "A beats B here" than at assigning a precise score to each answer—so pairwise labels are a natural data format even though the math later wants something like a scalar reward.

Example: composite reward (common in products)

$$R(x, y) \;=\; w_1 \cdot \underbrace{\mathbb{1}[\text{schema OK}]}_{\text{rules}} + w_2 \cdot \underbrace{s_{\text{help}}(x, y)}_{\text{e.g. preference model}} - w_3 \cdot \underbrace{\text{length penalty}}_{\text{brevity}}$$

Contrast with preference pairs: here you assume a **scalar** $R(x, y)$ for each completion—a single number you can add terms to, weight, and differentiate (often approximately). Optimization directly pushes probability toward completions with *higher* R, without needing a side-by-side comparison for every gradient step (though pairwise data may have been used earlier to *train* a component like s_{help}).

Components might be learned (preference model), hand-coded (regex, linter), or both. The engineering work is making R **align with real user value**—and not invite gaming (next slide).

Human intuition

Products rarely optimize one thing: you stack "must satisfy" checks, learned scores, and cheap penalties (length, latency). The weights are where product priorities become math.

What RL fine-tuning tries to maximize (sketch)

Let $R(x, y)$ be a (possibly stochastic) reward for prompt x and completion y. One canonical objective is expected reward under the policy:

$$J(\theta) \;=\; \mathbb{E}_{x\sim\mathcal{D}}\Big[\,\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[\,R(x,y)\,\big]\,\Big].$$

In words: draw realistic prompts, roll out the model, score outcomes, and push θ to put more mass on high-scoring answers. Practical algorithms also add a **KL penalty** toward a reference model so the policy does not collapse or forget capabilities.

Human intuition

Maximize "how good are the answers my model actually samples?"—not just how good one gold answer would be if you always copied it.

How does the gradient get through sampling? (REINFORCE idea)

For intuition, ignore KL for a moment. Write the probability of a trajectory under the policy as $\pi_\theta(\tau \mid x)$. A simple estimator-style picture:

$$\nabla_\theta \, \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] \;=\; \mathbb{E}_{\tau \sim \pi_\theta}\big[\, R(\tau) \, \nabla_\theta \log \pi_\theta(\tau \mid x) \,\big],$$

with the log-probability gradient factoring over tokens (sum of per-step terms).

Interpretation: if R is high, increase the log-probability of the actions you took; if R is low, decrease them. Variants subtract a **baseline** (advantage) to reduce variance—this is where much of the algorithmic complexity of PPO etc. lives.

Human intuition

Think of R as a thumbs-up strength: reinforce the path you sampled in proportion to how happy you are with the outcome—then add clever baselines so the signal is not pure noise.

Why a KL penalty to a reference model?

- The reward signal is often **incomplete** (preference model not perfect) or **sparse**.
- Without constraint, optimization can drift to high-R regions that are **unnatural** or **exploit quirks** of $\hat{R}$.
- A common fix: optimize something like

$$J(\theta) - \beta \cdot \mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big),$$

where π_{ref} is often the **SFT checkpoint**. β trades off alignment signal vs. staying close to something known-good.

Human intuition

The reward is usually imperfect; without a tether, optimization can drift into weird text that games the scorer. Staying near the SFT model is "don't wander off into a valley only the reward model thinks is paradise."

From preferences to training (two common paths)

1. **Reward model + RL:** fit $\hat{R}_\phi$ on human rankings, then run PPO / similar on $\hat{R}$ with KL penalty (classic RLHF picture).
2. **Direct preference optimization (DPO):** rewrite the RL objective under a Bradley–Terry preference model and update θ **without** explicit reward modeling or on-policy rollouts (implementation detail omitted—but the goal is the same: prefer chosen over rejected completions).

Both still rely on the same mental model: the **policy is next-token probabilities**; training reshapes those probabilities using preference or reward signals that SFT alone did not provide.

Reward hacking (still the core risk)

- The model maximizes *whatever* R measures, not what you *meant*.
- Examples: verbosity if length is not penalized, flattering but empty answers if a proxy "helpfulness" model likes tone, tool overuse if "called a tool" is rewarded.

Mitigations (high level)

Better rewards, adversarial evaluation, holdout human judgments, constrain with KL, and monitor in production—same story as any optimized system with a misspecified objective.

A common real-world pattern

1. Start with a strong base model
2. Apply **SFT** so the model can follow instructions and format (a stable π_{ref})
3. Apply **preference / RL-style training** to optimize richer objectives (helpfulness, safety, style) under KL
4. Evaluate continuously with realistic prompts and adversarial tests

Exact recipes vary by team, model family, and risk tolerance—but the sequencing is familiar across industry.

Outline

A practical adaptation pipeline (schematic)

1. **Define success:** what does “good” mean for users and for compliance?
2. **Collect / curate data:** examples, rubrics, and negative examples where applicable
3. **Train:** SFT and/or preference training with careful versioning
4. **Evaluate:** automatic metrics + human review + red teaming for serious deployments
5. **Deploy + monitor:** drift, regressions, abuse attempts, and data freshness

Evaluation

- **Offline eval:** fixed prompt sets, schema checks, unit tests for tools
- **Human eval:** sample real tasks; judge usefulness and policy adherence
- **Online eval:** A/B tests, user feedback, support escalations

Without evaluation, adaptation becomes storytelling: the demo looks good, the product does not.

Versioning and reproducibility

- Which **base model** checkpoint?
- which **adapter** weights and training configuration?
- which **dataset snapshot** (and license)?
- what **evaluation harness** gates a release?

These questions matter the moment adaptation leaves a notebook.

Outline

Demo goal

We'll show how **fine-tuning** can pull style from **examples** even when the **system prompt** says something different.

- **System prompt (baseline + eval):** answer as "Captain Cache," a playful pirate
- **SFT training labels:** casual, Gen Z–inflected internet English (classroom-safe)
- **Method:** supervised fine-tuning with LoRA (via a training API)
- **Before / after:** same pirate prompt—compare base model vs adapter on fresh questions

This is intentionally a toy—real deployments need more data, stronger eval, and clearer safety constraints.

Outline

Data, licensing, safety, and misuse

Data and licensing

- Training data may include sensitive information—treat curation like a privacy and security process.
- Respect model licenses and dataset licenses; “open weights” does not mean “no rules.”

Safety and misuse

- Style fine-tuning can make outputs **more persuasive**; combine with policies and monitoring.
- Adapters can be swapped; treat serving endpoints as security-sensitive if they execute actions.
- For customer-facing systems, plan for adversarial prompts and abuse scenarios.

Closing checklist (for your next project)

1. Can we state the desired behavior as **tests** and **rubrics**?
2. Is prompting enough, or do we need **SFT** for consistency?
3. If we train: what is our **dataset**, and who owns it?
4. Are we using **LoRA** (or similar) to control cost and iteration speed?
5. Do we need **preference / RL-style** training, and what is the reward?
6. What is our **evaluation** plan before and after launch?

Thank you

Questions?

Demo: `examples/model_adaptation/README.md`